



Object Oriented Programming



Lecture#2 Classes and Objects

ภาษาเชิงกระบวนการ และภาษาเชิงวัตถุ

ภาษาเชิงกระบวนการ	ภาษาเชิงวัตถุ
☐ โปรแกรมจะแบ่งออกเป็นส่วนย่อย ๆ ที่เรียกว่าโมดูล(module)	☐ สามารถนำโปรแกรมกลับมาใช้ใหม่ได้ดีกว่าภาษาเชิงกระบวนการ
☐ การออกแบบให้แต่ละโมดูลมีความเป็นอิสระต่อกันทำได้ยาก	☐ การพัฒนาโปรแกรมเป็นการเลียนแบบการทำงานเชิงวัตถุ
☐ ต้นทุนในการพัฒนาโปรแกรมสูง	
☐ แต่ละโมดูลจะต้องเป็นอิสระต่อกัน	



การพัฒนาโปรแกรมเชิงวัตถุ

- ▶ มองปัญหาว่าประกอบไปด้วยออบเจ็คต่างๆ ซึ่งแนวคิดนี้จะเข้าใจใกล้เคียงกับธรรมชาติของมนุษย์มากที่สุดเนื่องจากมนุษย์มองสิ่งต่างๆ รอบตัวเป็นออบเจ็คทั้งที่เป็นรูปธรรม (วัตถุ) เช่น ปากกา นักศึกษา หรือใบลงทะเบียน เป็นต้น และที่เป็นนามธรรมเช่น คะแนน หรือรายชื่อวิชา เป็นต้น
- ▶ การเขียนโปรแกรมเชิงออบเจ็คจะเป็นขบวนการการวิเคราะห์ปัญหา โดยการจำลองคุณลักษณะและพฤติกรรมของออบเจ็คในระบบจริง ให้อยู่ในรูปของโปรแกรมคอมพิวเตอร์ ตัวอย่างเช่น
- ▶ การพัฒนาโปรแกรมระบบทะเบียนนักศึกษาอาจแบ่งโปรแกรมให้ประกอบด้วยออบเจ็คต่างๆ เช่น นักศึกษา ใบลงทะเบียน และรายวิชา เป็นต้น
 - ▶ ออบเจ็คชนิดนักศึกษาอาจมีคุณลักษณะต่างๆ เช่น ชื่อ รหัส และเกรดเฉลี่ย เป็นต้น และอาจมีพฤติกรรมที่นักศึกษาสามารถกระทำได้เช่น ลงทะเบียน และเพิ่มหรือถอนวิชา เป็นต้น



ระบบทะเบียนนักศึกษา

วิธีการเชิงกระบวนการ

☐ ลงทะเบียนรายวิชา

☐ ชำระเงิน

☐ เพิ่มวิชา

วิธีแบบเชิงออกแบบ

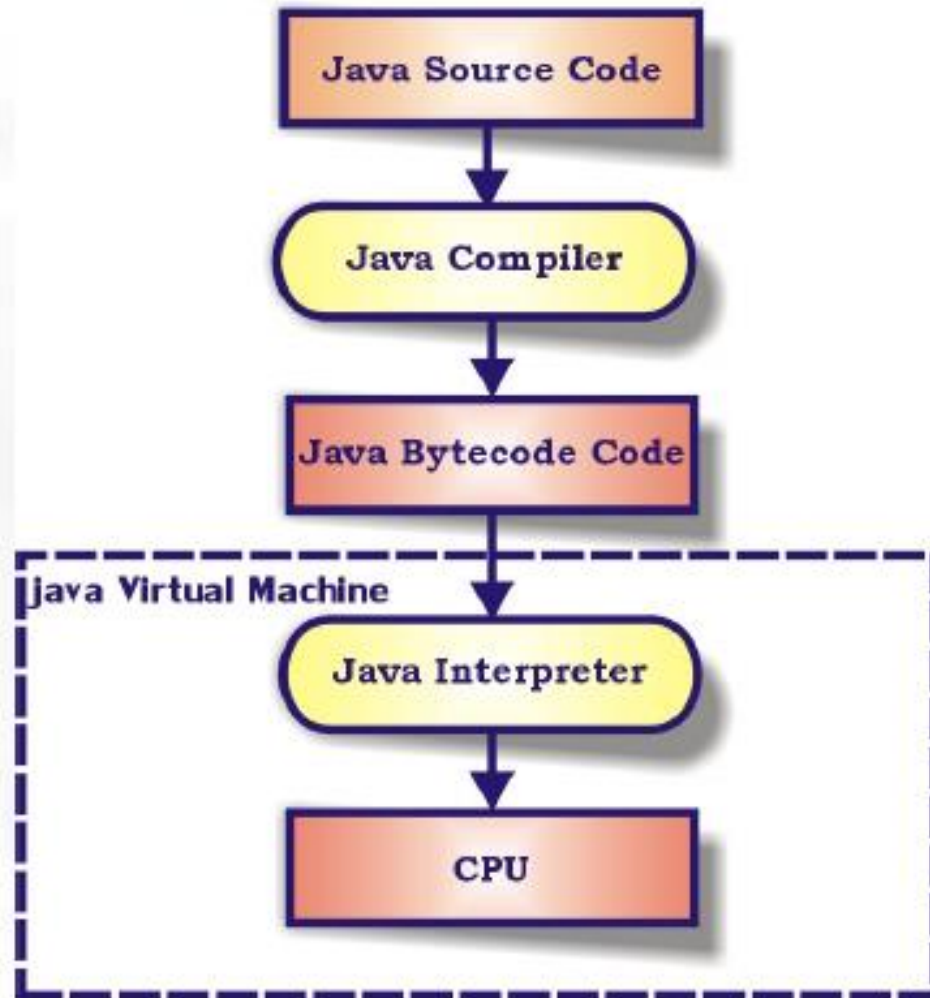
☐ นักศึกษา

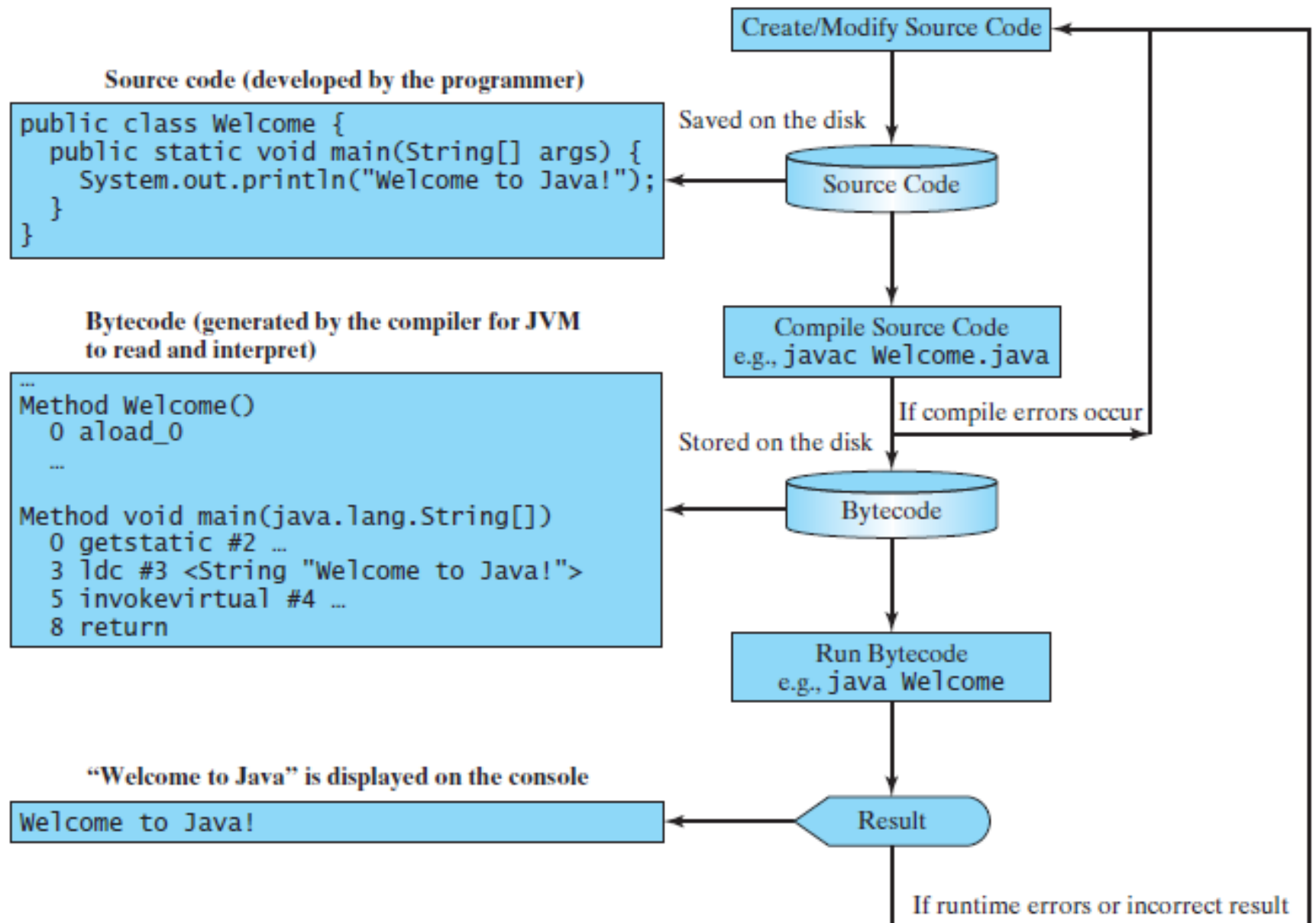
☐ ใบลงทะเบียน

☐ รายชื่อวิชา



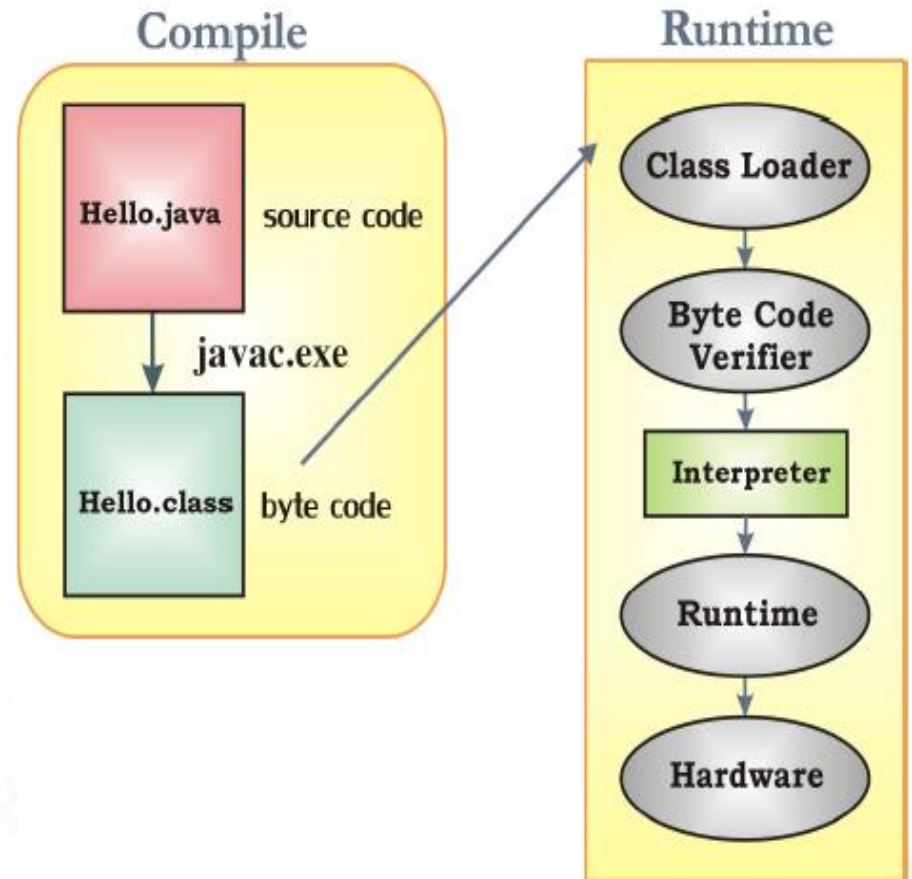
ขั้นตอนการทำงานของโปรแกรมภาษาจาวา





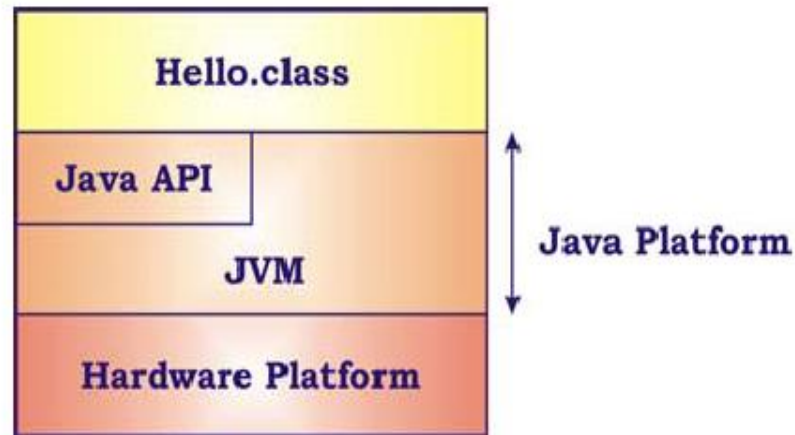
JRE(Java Runtime Environment)

- ▶ JRE จะรันโปรแกรมที่แปลมาจาก JVM โดยจะทำงาน 3 ขั้นตอนคือ
 - ▶ โหลดไบต์โค้ดโดยใช้ Class Loader
 - ▶ ตรวจสอบไบต์โค้ดโดยใช้ Byte Code Verifier
 - ▶ รันไบต์โค้ดโดยใช้ Runtime Interpreter



แพลตฟอร์มของเทคโนโลยีจาวา

- ▶ แพลตฟอร์ม (Platform) คือฮาร์ดแวร์และสภาวะแวดล้อมทางซอฟต์แวร์ (Software Environment) ที่โปรแกรมจะใช้ในการประมวลผลโดยทั่วไป
- ▶ แพลตฟอร์มของเทคโนโลยีจาวาจะประกอบด้วย
 - ▶ Java Virtual Machine (JVM)
 - ▶ Java Application Programming Interface (Java API)



การพัฒนาโปรแกรมโดยใช้ภาษาคอมไพเตอร์เชิงออปเจ็ค

- ▶ การพัฒนาโปรแกรมโดยใช้ภาษาคอมไพเตอร์เชิงออปเจ็ค จะทำให้ขอบข่ายการพัฒนาโปรแกรมทำได้รวดเร็วขึ้นและสามารถปรับปรุงแก้ไขโปรแกรมได้ง่าย ซึ่งเหมาะกับการพัฒนาโปรแกรมขนาดใหญ่ที่จะต้องมีการปรับปรุงแก้ไขโปรแกรมอยู่ตลอด นอกจากนี้โปรแกรมเชิงออปเจ็คยังมีคุณลักษณะเด่นอื่นๆ อีกดังนี้
 - ▶ การห่อหุ้ม (Encapsulation) เป็นคุณลักษณะที่ทำให้ออปเจ็คแต่ละตัวเป็นอิสระต่อกัน ซึ่งทำให้สามารถแบ่งการพัฒนาโปรแกรมออกเป็นส่วนๆ ได้ง่าย
 - ▶ การสืบทอด (Inheritance) เป็นคุณลักษณะที่ทำให้สามารถนำโปรแกรมที่พัฒนาแล้วกลับมาใช้ใหม่ได้ง่ายกว่าการเขียนโปรแกรมเชิงกระบวนการ
 - ▶ การมีได้หลายรูปแบบ (Polymorphism) เป็นคุณลักษณะที่ทำให้นักพัฒนาโปรแกรมสามารถเพิ่มเติมส่วนต่างๆ ของโปรแกรมได้ง่าย



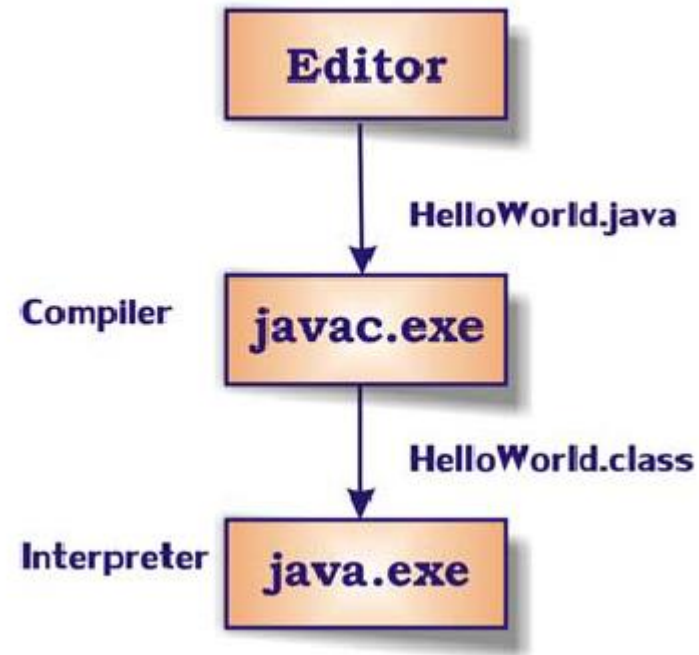
Create, Compiling, and Executing a Java Program

- ▶ โปรแกรมจาวาที่อยู่บนแพลตฟอร์ม J2SE สามารถพัฒนาได้สองรูปแบบคือ
 - ▶ 1. โปรแกรมจาวาประยุกต์ (Java Application) คือโปรแกรมประยุกต์ใช้งานทั่วไปโดยโปรแกรมแบบนี้จะทำงานภายใต้โปรแกรมอินเตอร์พรีตเตอร์โดยตรง ซึ่งโปรแกรมลักษณะนี้เป็น stand-alone
 - ▶ 2. โปรแกรมจาวาแอปเพล็ต (Java Applet) คือโปรแกรมภาษาจาวาที่จะทำงานภายใต้โปรแกรมเว็บเบราว์เซอร์ที่มี JVM อยู่



การสร้างโปรแกรมจาวาประยุกต์

- ▶ เขียนซอร์สโค้ด -> HelloWorld.java
- ▶ คอมไพล์โปรแกรม
 - ▶ `javac HelloWorld.java`
- ▶ รันโปรแกรม
 - ▶ `java HelloWorld`



โครงสร้างของโปรแกรมภาษา Java

Classes

Modifiers

Methods

Comments

Reserved word

```
public class ComputeArea{  
    public static void main(String[] args){  
        // Compute the first area  
        double radius = 1.0;  
        double area = radius*radius*3.14159;  
        System.out.println("The area is" + area + "for radius" +radius);  
        // compute the second area  
        radius = 2.0;  
        area = radius*radius*3.14159;  
        System.out.println("The area is" + area + " for radius" +radius);  
    }  
}
```

Blocks

Statement

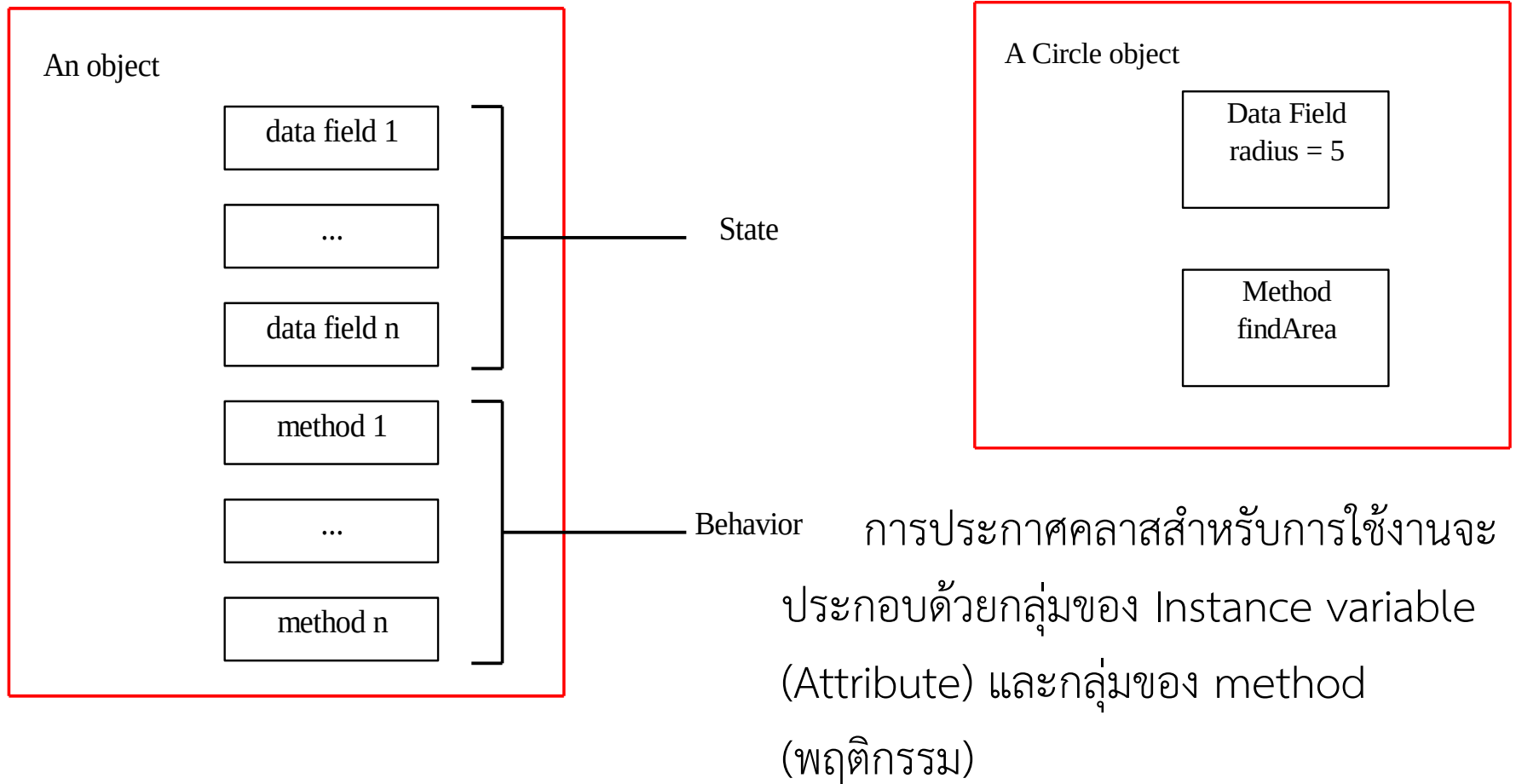


นิยามและความหมายของคลาสและวัตถุ

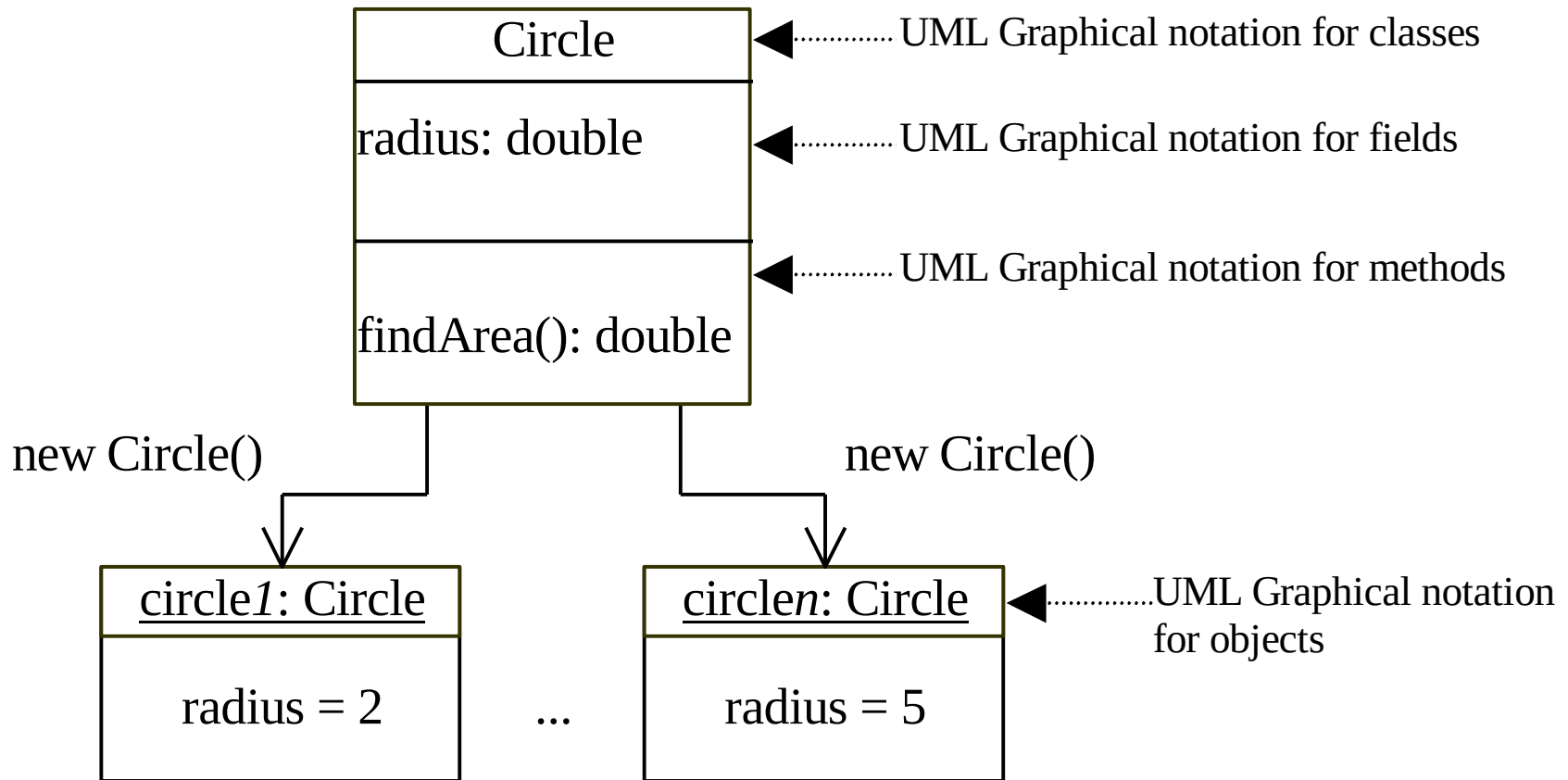
- ▶ **ออปเจกต์หรือวัตถุ(Object)** คือสิ่งต่างๆที่มีอยู่ในชีวิตประจำวันแบ่งได้เป็นสองประเภทคือ
 - ▶ 1. สิ่งที่เป็นรูปธรรม (Tangible) คือสิ่งที่เป็นวัตถุและจับต้องได้ เช่น นักศึกษา ใบลงทะเบียน ปากกา และรถ เป็นต้น
 - ▶ 2. สิ่งที่เป็นนามธรรม (Intangible) คือสิ่งที่ไม่สามารถจับต้องได้ เช่น คะแนน รายชื่อวิชา บัญชี เงินฝาก และตารางเที่ยวบิน เป็นต้น
- ▶ โดยส่วนใหญ่ออปเจกต์ต่างๆจะประกอบไปด้วยคุณลักษณะ (Attribute) และพฤติกรรม (behavior) โดยที่
 - ▶ คุณลักษณะก็คือข้อมูลของออปเจกต์
 - ▶ พฤติกรรมหรือเรียกว่าเมธอด (method) คือสิ่งที่ ออปเจกต์สามารถกระทำได้ ซึ่งในโปรแกรมเชิงวัตถุก็คือคำสั่งในการทำงาน
- ▶ โปรแกรมเชิงวัตถุจะประกอบด้วยออปเจกต์ต่างๆหลายออปเจกต์ ซึ่งแต่ละออปเจกต์จะมีคุณลักษณะต่างๆที่เป็นข้อมูลของวัตถุเอง



OO Programming Concepts



Class and Objects



Methods

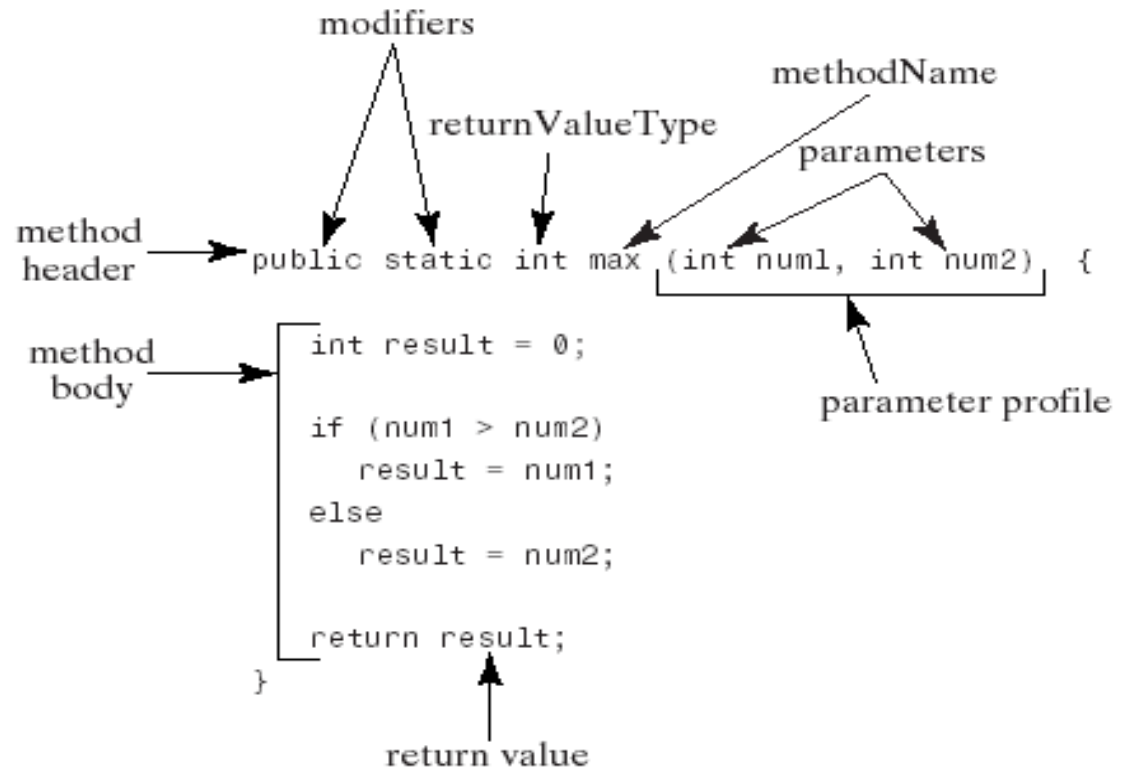
- ▶ เป็นชุดคำสั่งย่อยที่ทำงานกับข้อมูล ตัวเมธอดอาจจะมีหรือไม่มีพารามิเตอร์ก็ได้ โดยที่เมธอดอาจจะมีค่าคืนค่าออกมาภายหลังจากที่ทำงานเสร็จ
- ▶ ประเภทของเมธอด ประกอบด้วย
 - ▶ Mutator methods (setter) ทำหน้าที่เปลี่ยนข้อมูลภายในวัตถุ
 - ▶ Accessor methods (getter) ทำหน้าที่ดึงข้อมูลจากวัตถุ
 - ▶ เมธอดอื่น ๆ



Introducing Methods

เป็นชุดคำสั่ง
ย่อยที่ทำงานกับ
ข้อมูล

Method Structure



การประกาศเมธอด

```
public static int max(int num1, int num2)
{
    if (num1 > num2)
        return num1;
    else
        return num2;
}
```

static สามารถเรียกใช้งานโดยไม่จำ ว่าเป็นต้องสร้าง object ขึ้นมาก่อน

→ Class method



ชนิดของ **method**

- ▶ method ในภาษา Java มี 2 ประเภท ดังนี้
 - ▶ 1. method ที่เป็นสมาชิกของ class (class method):
 - ▶ method ประเภทนี้จะมีเพียงชุดเดียวเท่านั้น และไม่จำเป็นต้องสร้าง object เพื่อเรียกใช้ method
 - ▶ method นี้สามารถเรียกใช้ผ่านชื่อ class ได้
 - ▶ access_modifier ที่ทำให้ method เป็นสมาชิกของ class คือ **static** method ประเภทนี้เหมาะสำหรับ method ที่ไม่มีการเปลี่ยนแปลงค่าไปตาม object เช่น สูตรในการคำนวณทางคณิตศาสตร์ ซึ่ง method ดังกล่าวจะรับค่าที่ผู้ใช้ต้องการคำนวณ และทำการคำนวณแล้วส่งผลลัพธ์กลับไปยังผู้ใช้
 - ▶ 2. method ที่เป็นสมาชิกของ instance (instance method):
 - ▶ method ที่เป็นสมาชิกของ instance จะต้องเกิดจากตัวดำเนินการ new กล่าวคือ เมื่อจะเรียกใช้ method ประเภทนี้ จะต้องมีการสร้าง object ขึ้นมาก่อน และค่าของข้อมูลจะเปลี่ยนไปตาม object ที่สร้างขึ้น

ชนิดของ **method**

Class method	Instance method
<pre>class X { public static int methodA(int a) { a += 1; return a; } } class TestClassMember { public static void main (String args[]) { int num = 0; num = num + X.methodA(num); System.out.println("num="+num); } }</pre>	<pre>class X { int methodA(int a) { a += 1; return a; } } class TestClassMember1 { public static void main (String args[]) { int num = 0; X numX = new X(); num = num + numX.methodA(num); System.out.println("num="+num); } }</pre>

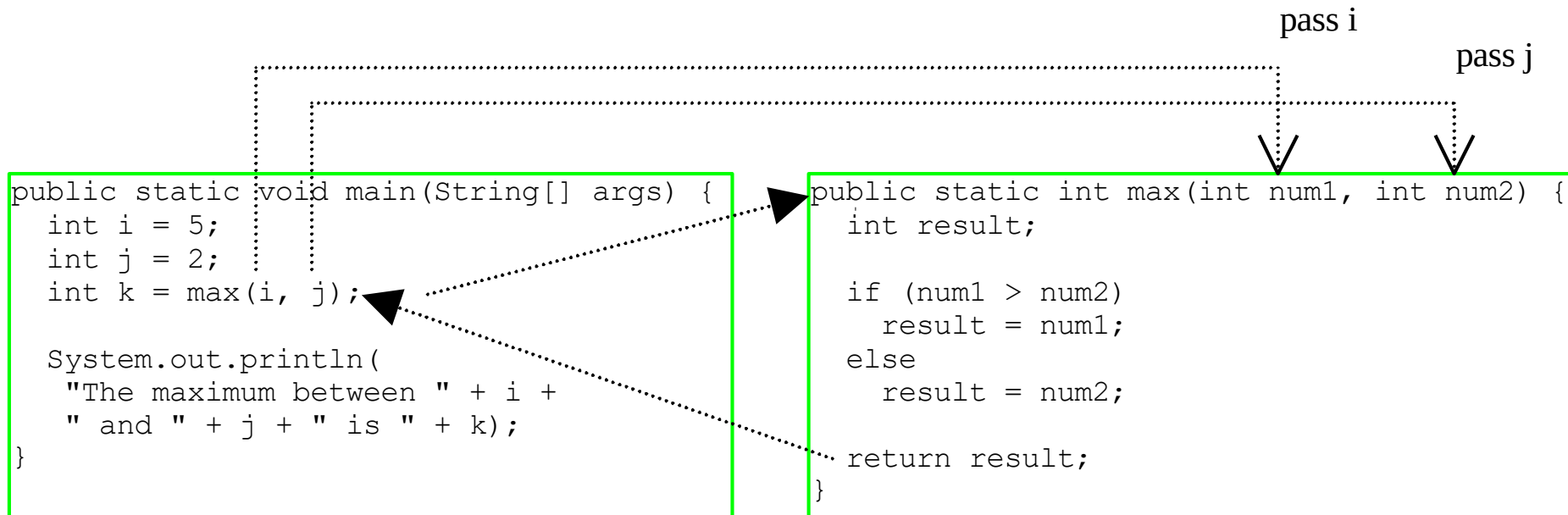
- ▶ class method ต้องมี modifier แบบ static และสามารถเรียกประมวลผลผ่านชื่อของ class (X.methodA();)
- ▶ instance method ไม่ต้องมี modifier เป็น static แต่ก่อนที่จะเรียกใช้งานจะต้องมีการสร้าง object ขึ้นมาก่อน (X numX = new X();) และการเรียกประมวลผลต้องเรียกผ่านชื่อของ object นั้น (numX.methodA();)
- ▶ การเรียกใช้ class method ที่อยู่ภายใน class เดียวกัน สามารถที่จะเรียกใช้ผ่านชื่อ method นั้นได้เลย

Calling Methods

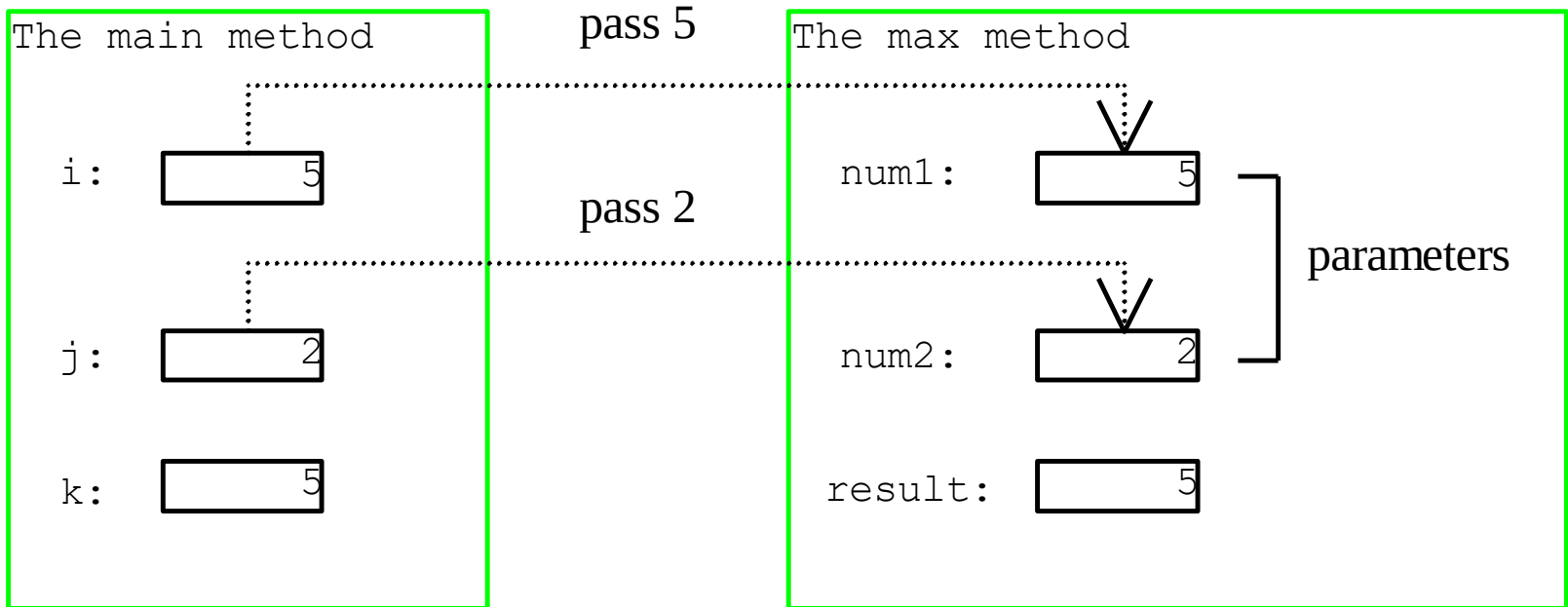
```
public class TestMax {  
    /** Main method */  
    public static void main(String[] args) {  
        int i = 5;  
        int j = 2;  
        int k = max(i, j);  
        System.out.println("The maximum between " + i +  
            " and " + j + " is " + k);  
    }  
  
    /** Return the max between two numbers */  
    public static int max(int num1, int num2) {  
        int result;  
  
        if (num1 > num2)  
            result = num1;  
        else  
            result = num2;  
  
        return result;  
    }  
}
```



Calling Methods



Calling Methods



Fields

▶ 필드อาจเรียกอีกอย่างหนึ่งคือ Attributes หรือ Data members ประกอบด้วย

▶ instance variables

▶ Class variables



Class Declaration

ตัวอย่างการประกาศคลาส Circle ที่ประกอบด้วยฟิลด์ radius และเมธอด findArea()

```
class Circle {  
    double radius = 1.0;  
  
    double findArea() {  
        return radius * radius * 3.14159;  
    }  
}
```



การประกาศตัวแปรอ้างอิงวัตถุ (Declaring Object Reference Variables)

- ▶ รูปแบบการประกาศตัวแปรอ้างอิงวัตถุมีรูปแบบคือ

`ClassName objectReference;`

- ▶ ตัวอย่าง

`Circle myCircle;`

- ▶ การสร้างวัตถุ (Creating Objects)

- ▶ ภายหลังการประกาศตัวแปรอ้างอิงวัตถุแล้วหลังจากนั้นจะสร้างวัตถุภายในหน่วยความจำโดยใช้คำสั่ง `new` และตามด้วยคอนสตรัคเตอร์ที่จะใช้สร้างวัตถุ เช่น `new ClassName()` และเก็บที่อยู่ดังกล่าวไว้ในตัวแปรอ้างอิงวัตถุที่ประกาศไว้ก่อนหน้านี้ โดยมีรูปแบบดังนี้

`objectReference = new ClassName();`

- ▶ ตัวอย่าง

`myCircle = new Circle();`

- ▶ วัตถุที่ถูกอ้างอิง(Object reference) จะถูกกำหนดให้กับตัวแปรอ้างอิงวัตถุ (object reference variable) เช่น `myCircle` เป็นตัวแปรที่เก็บที่อยู่ของวัตถุที่สร้างจากคลาส `Circle`



การประกาศวัตถุและการสร้างวัตถุในขั้นตอนเดียว

```
ClassName objectReference = new ClassName();
```

ตัวอย่าง

```
Circle myCircle = new Circle();
```



รูปแบบการทำงานที่แตกต่างกันของตัวแปรอ้างอิงวัตถุ (Object Types) และ ตัวแปรของชนิดข้อมูลธรรมดา (Primitive Data Types)

Primitive type

int i = 1

i

1

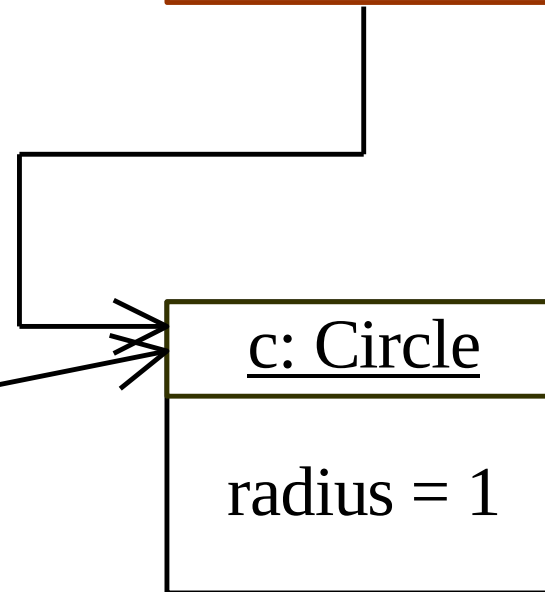
Object type

Circle c

c

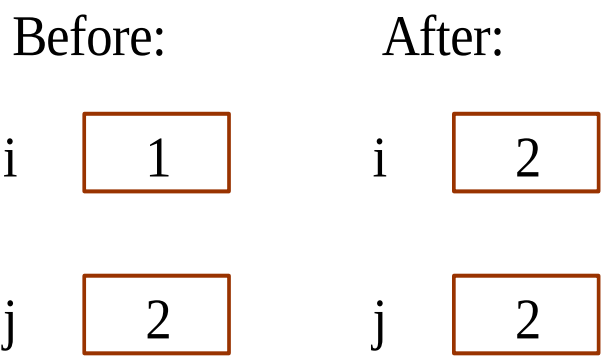
reference

Created using
new Circle()

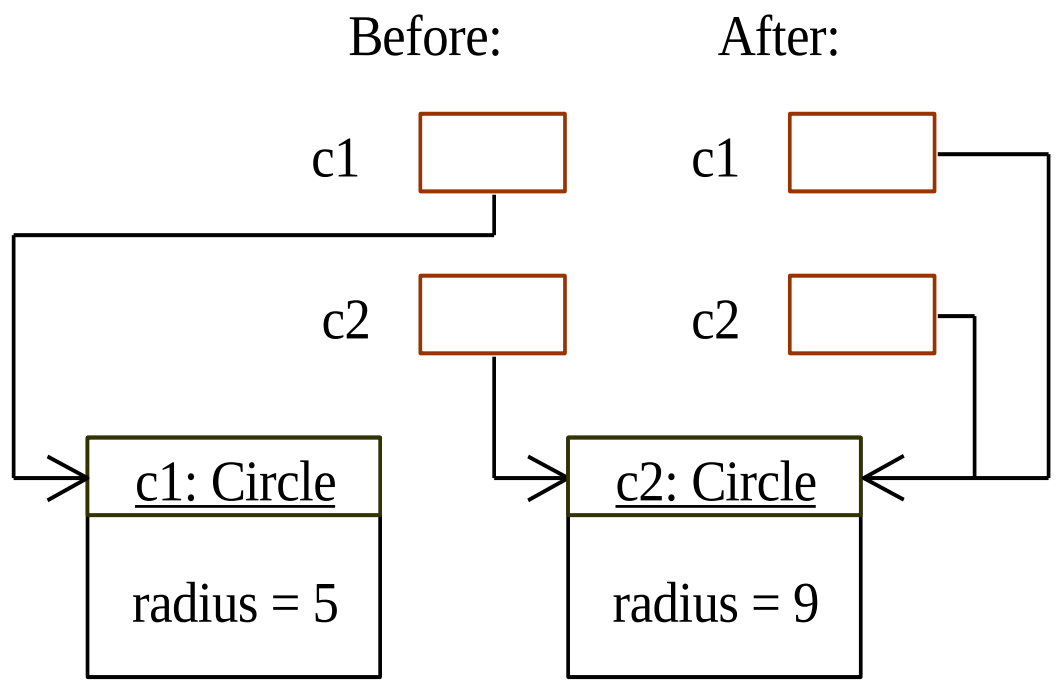


การคัดลอกข้อมูลของ Primitive Data Types and Object Types

Primitive type assignment
`i = j`



Object type assignment
`c1 = c2`



Garbage Collection

- จากรูปก่อนหน้านี้จะพบว่า
 - หลังจากกำหนดให้ $c1 = c2$ จะพบว่า $c1$ ชี้ไปยังตำแหน่งเดียวกับ $c2$
 - จากเหตุการณ์ดังกล่าวจะพบว่า $c1$ จะไม่ถูกใช้ซึ่งทำให้วัตถุดังกล่าวกลายเป็นขยะหรือ garbage
 - เมื่อเกิดเหตุการณ์ดังกล่าว garbage จะถูกเก็บโดย JVM ซึ่งก็คือการคืนหน่วยความจำที่ใช้นั่นเอง



การเข้าถึงข้อมูลของวัตถุ (Accessing Objects)

- ▶ การอ้างอิงแอตทริบิวต์หรือข้อมูลภายในวัตถุสามารถทำได้โดยใช้รูปแบบดังต่อไปนี้

`objectReference.data`

- ▶ เช่น

`myCircle.radius;`

- ▶ การอ้างอิงเมธอดภายในวัตถุสามารถทำได้โดยใช้รูปแบบดังต่อไปนี้

`objectReference.method`

- ▶ เช่น

`myCircle.findArea();`



Example Using Objects

คลาส Circle3

```
public class Circle3 {
    /** The radius of the circle */
    private double radius = 1;
    /** The number of the objects created */
    private static int numberOfObjects = 0;
    /** Construct a circle with radius 1 */
    public Circle3() {
        numberOfObjects++;
    }
    /** Construct a circle with a specified radius */
    public Circle3(double newRadius) {
        radius = newRadius;
        numberOfObjects++;
    }
    /** Return radius */
    public double getRadius() {
        return radius;
    }
    /** Set a new radius */
    public void setRadius(double newRadius) {
        radius = (newRadius >= 0) ? newRadius : 0;
    }
    /** Return numberOfObjects */
    public static int getNumberOfObjects() {
        return numberOfObjects;
    }
    /** Return the area of this circle */
    public double getArea() {
        return radius * radius * Math.PI;
    }
}
```



Example Using Objects

คลาส TestCircle3

```
▶ public class TestCircle3 {  
    /** Main method */  
    public static void main(String[] args) {  
        // Create a Circle with radius 5.0  
        Circle3 myCircle = new Circle3(5.0);  
        System.out.println("The area of the circle of radius "  
            + myCircle.getRadius() + " is " + myCircle.getArea());  
        // Increase myCircle's radius by 10%  
        myCircle.setRadius(myCircle.getRadius() * 1.1);  
        System.out.println("The area of the circle of radius " +  
            myCircle.getRadius() + " is " + myCircle.getArea());  
    }  
}
```



คอนสตรัคเตอร์ **Constructors**

- ▶ คอนสตรัคเตอร์เป็นเมธอดชนิดหนึ่งที่มีชื่อเดียวกับชื่อคลาส
- ▶ ถูกเรียกเพื่อสร้างวัตถุและกำหนดค่าเริ่มต้นให้กับวัตถุ
- ▶ คอนสตรัคเตอร์ที่ไม่มีพารามิเตอร์จะเรียกว่า *Default constructor*
- ▶ คอนสตรัคเตอร์จะต้องมีชื่อเดียวกับชื่อคลาส
- ▶ การสร้างคอนสตรัคเตอร์จะไม่มีส่วนของการคืนค่ากลับ
- ▶ หน้าที่ของคอนสตรัคเตอร์คือการกำหนดค่าเริ่มต้นให้กับวัตถุ (initializing objects)
- ▶ เมื่อต้องการสร้างวัตถุจะเรียกใช้คอนสตรัคเตอร์พร้อมกำหนดค่าเริ่มต้นให้กับวัตถุในคอนสตรัคเตอร์ เช่น

```
myCircle = new Circle(5.0);
```



คอนสตรัคเตอร์ **Constructors**

```
Circle(double r) {  
    radius = r;  
}
```

```
Circle() {  
    radius = 1.0;  
}
```

```
myCircle = new Circle(5.0);
```

Constructors are a special kind of methods that are invoked to construct objects.



คอนสตรัคเตอร์ **Constructors**

- ▶ คอนสตรัคเตอร์เป็นโปรแกรมย่อยสำหรับกำหนดค่าเริ่มต้นให้แก่วัตถุ มีคุณสมบัติพิเศษดังนี้ คือ
 - ▶ 1. จะถูกดำเนินการเมื่อมีการสร้างวัตถุเสมอไม่สามารถละเลย หรือหลงลืมได้ ทุกครั้งที่มีการเรียกคำสั่ง `new` จะต้องมีการเรียกใช้คอนสตรัคเตอร์เสมอ
 - ▶ 2. ต้องมีชื่อเมธอดเหมือนกับชื่อคลาส ถ้าเราลองสังเกตให้ดีหลังคำสั่ง `new` จะมีชื่อคลาสแล้วตามด้วยวงเล็บ
 - ▶ 3. การประกาศโปรแกรมย่อยคอนสตรัคเตอร์ไม่ต้องระบุ `void` และไม่ต้องระบุ `return type` เช่นรูปแบบการประกาศคอนสตรัคเตอร์แบบดีฟอลท์



Example

```
▶ class Student {  
    String id;  
    String name;  
    short startYear;  
    float gpa = 0.0F;  
    Student() {  
    }  
    void print() {  
        System.out.println("id : "+ id);  
        System.out.println("name : "+ name);  
        System.out.println("year : "+ startYear);  
        System.out.println("gpa : "+ gpa);  
        System.out.println("=====");  
    }  
}
```



คอนสตรัคเตอร์ Constructors

- ▶ หรือเราสามารถประกาศคอนสตรัคเตอร์ที่มีพารามิเตอร์ตามที่ต้องการ แทนที่คอนสตรัคเตอร์แบบดีฟอลท์โดยการประกาศพารามิเตอร์และกำหนดค่าให้กับตัวแปรได้ เช่น

```
class Student {
    String id;
    String name;
    short startYear;
    float gpa = 0.0F;
    Student() {
    }
    Student(String newId, String newName, short newStartYear) {
        id = newId;
        name = newName;
        startYear = newStartYear;
    }
    void print() {
        System.out.println("id : "+ id);
        System.out.println("name : "+ name);
        System.out.println("year : "+ startYear);
        System.out.println("gpa : "+ gpa);
        System.out.println("=====");
    }
}

class TestStudent {
    public static void main(String[] args) {
        Student std1;
        std1 = new Student("46F0000", "Tom", 2546);
        Student std2;
        std2 = new Student("46F0999", "Jerry", 2546);
        std1.print();
        std2.print();
    }
}
```

คอนสตรัคเตอร์ Constructors

- ▶ เมื่อเราประกาศคอนสตรัคเตอร์ของตัวเองขึ้นมาจะทำให้ไม่สามารถเรียกใช้คอนสตรัคเตอร์แบบดีฟอลต์ได้อีกต่อไป ทำให้การสร้างวัตถุจากคลาส Student ทุกครั้งจะต้องระบุ พารามิเตอร์ รหัสและชื่อนักศึกษาอย่างครบถ้วน หากไม่ใส่พารามิเตอร์หรือไม่ครบหรือผิดประเภทคอมไพเลอร์จะตรวจสอบได้และแจ้งข้อผิดพลาดให้เราทราบ เมื่อประกาศคอนสตรัคเตอร์ภายในคอนสตรัคเตอร์ควรมีการกำหนดค่าเริ่มต้นให้กับตัวแปรของวัตถุให้ครบทุกตัวตอนสร้างวัตถุ

```
▶ Student(String newId, String newName, short newStartYear) {  
    id = newId;  
    name = newName;  
    startYear = newStartYear;  
}
```

Ex- Class Employee

```
class Employee {  
    // constructors  
  
    // methods  
  
    // fields  
  
}
```

Employee
-name: String -location: String -salary: double
+Employee(name : String, location : String, salary : double) +getName() : String +getLocation() : String +getSalary() : double +setName(name : String) +setLocation(location : String) +setSalary(salary : double) +increaseSalary(percentage : double) +toString() : String



Class Employee -Fields

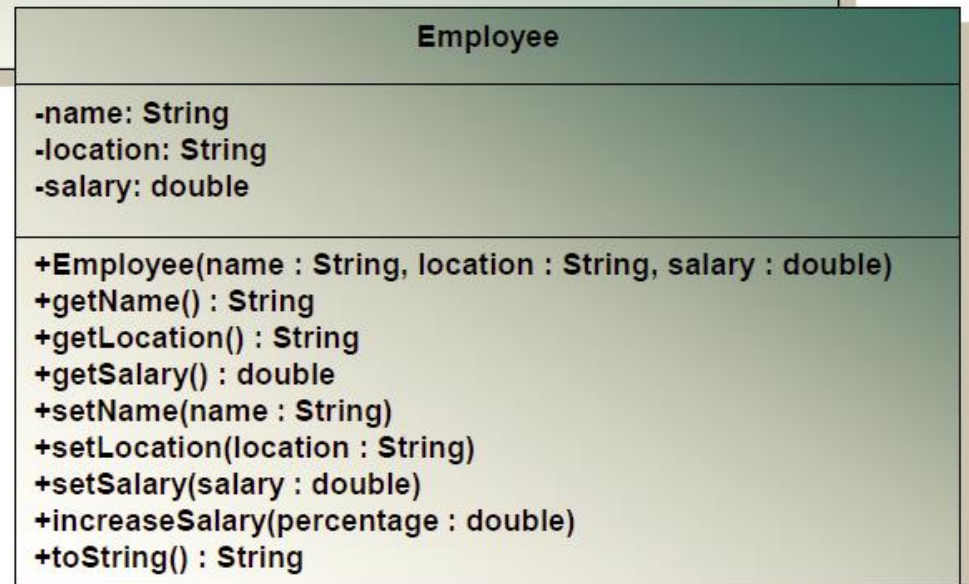
```
private String name;  
private String location;  
private double salary;
```

Employee
-name: String -location: String -salary: double
+Employee(name : String, location : String, salary : double) +getName() : String +getLocation() : String +getSalary() : double +setName(name : String) +setLocation(location : String) +setSalary(salary : double) +increaseSalary(percentage : double) +toString() : String



Class Employee -Constructor

```
public Employee(String name, String location, double salary){  
    this.name = name;  
    this.location = location;  
    this.salary = salary;  
}
```



Class Employee –Accessor Methods

```
public String getName() {  
    return name;  
}  
  
public String getLocation() {  
    return location;  
}  
  
public double getSalary() {  
    return salary;  
}
```

Employee
-name: String -location: String -salary: double
+Employee(name : String, location : String, salary : double) +getName() : String +getLocation() : String +getSalary() : double +setName(name : String) +setLocation(location : String) +setSalary(salary : double) +increaseSalary(percentage : double) +toString() : String

Class Employee –Mutator Methods

```
public void setName(String name) {  
    this.name = name;  
}  
  
public void setLocation(String location) {  
    this.location = location;  
}  
  
public void setSalary(double salary) {  
    this.salary = salary;  
}
```

Employee
-name: String -location: String -salary: double
+Employee(name : String, location : String, salary : double) +getName() : String +getLocation() : String +getSalary() : double +setName(name : String) +setLocation(location : String) +setSalary(salary : double) +increaseSalary(percentage : double) +toString() : String

Class Employee –Other Methods

```
public void increaseSalary(double percentage) {  
    salary = salary*(1 + percentage);  
}  
  
public String toString() {  
    return "Name : " + name +  
        ", Location : " + location +  
        ", salary : " + salary;  
}
```

Employee
-name: String -location: String -salary: double
+Employee(name : String, location : String, salary : double) +getName() : String +getLocation() : String +getSalary() : double +setName(name : String) +setLocation(location : String) +setSalary(salary : double) +increaseSalary(percentage : double) +toString() : String

ระดับการเข้าถึงวัตถุ(Visibility Modifiers and Accessor Methods)

- ▶ สรุปคำที่ใช้เขียนกำกับการประกาศคุณสมบัติหรือเมธอดภายในวัตถุเพื่อกำหนดระดับการเข้าถึงวัตถุทั้งหมด (Visibility Modifier) ประกอบด้วย
 - ▶ 1. ระดับ default เป็นระดับการเข้าถึงโดยปริยาย เมื่อวัตถุมีการประกาศคุณสมบัติและเมธอด โดยไม่กำหนดระดับการเข้าถึง เมื่อมีการกำหนดดังกล่าว คลาส ตัวแปรหรือข้อมูลสามารถที่จะเข้าถึงได้โดยคลาสทุก ๆ คลาสที่อยู่ใน package เดียวกัน
 - ▶ 2. ระดับ private เป็นระดับการเข้าถึงเมื่อมีการประกาศ โดยคำว่า private เมื่อมีการกำหนดดังกล่าว คลาส ตัวแปร ข้อมูล หรือ เมธอดสามารถจะเข้าถึงได้เฉพาะภายในคลาส การเข้าถึงข้อมูลดังกล่าวต้องทำผ่านเมธอด get หรือ set เท่านั้น
 - ▶ 3. ระดับ protected เป็นระดับการเข้าถึงเมื่อมีการประกาศ โดยคำว่า protected การอ้างถึงต้องอยู่ในรูปแบบของ this หรือ super เท่านั้น
 - ▶ 4. ระดับ public เป็นระดับการเข้าถึง เมื่อมีการประกาศ ด้วยคำว่า public การกำหนดคำดังกล่าวกับคลาส ตัวแปร ข้อมูล หรือ เมธอด จะทำให้คลาส ตัวแปร ข้อมูลหรือเมธอดดังกล่าวสามารถที่จะเข้าถึงได้โดยคลาสทุก ๆ คลาสที่อยู่ใน package เดียวกันหรือต่าง package

► ตัวอย่าง การใช้เมธอด get และ set

```
public class Person {  
    private String name;  
    private Date birthdate;  
    public Person(String name, Date birthdate) {  
        this.name = name;  
        this.birthdate = birthdate;  
    }  
    public String getName() {  
        return name;  
    }  
    public Date getBirthdate() {  
        return birthdate;  
    }  
}
```



Passing Objects to Methods

- ▶ การส่งค่าไปยังเมธอดจะเป็นการส่งโดยค่า (Passing by value) ซึ่งค่าที่ส่งไปจะเป็นค่าที่อ้างอิงไปยังวัตถุ ตัวอย่างของการส่งค่าวัตถุเป็นอาร์กิวเมนต์
- ▶ Example Passing Objects as Arguments

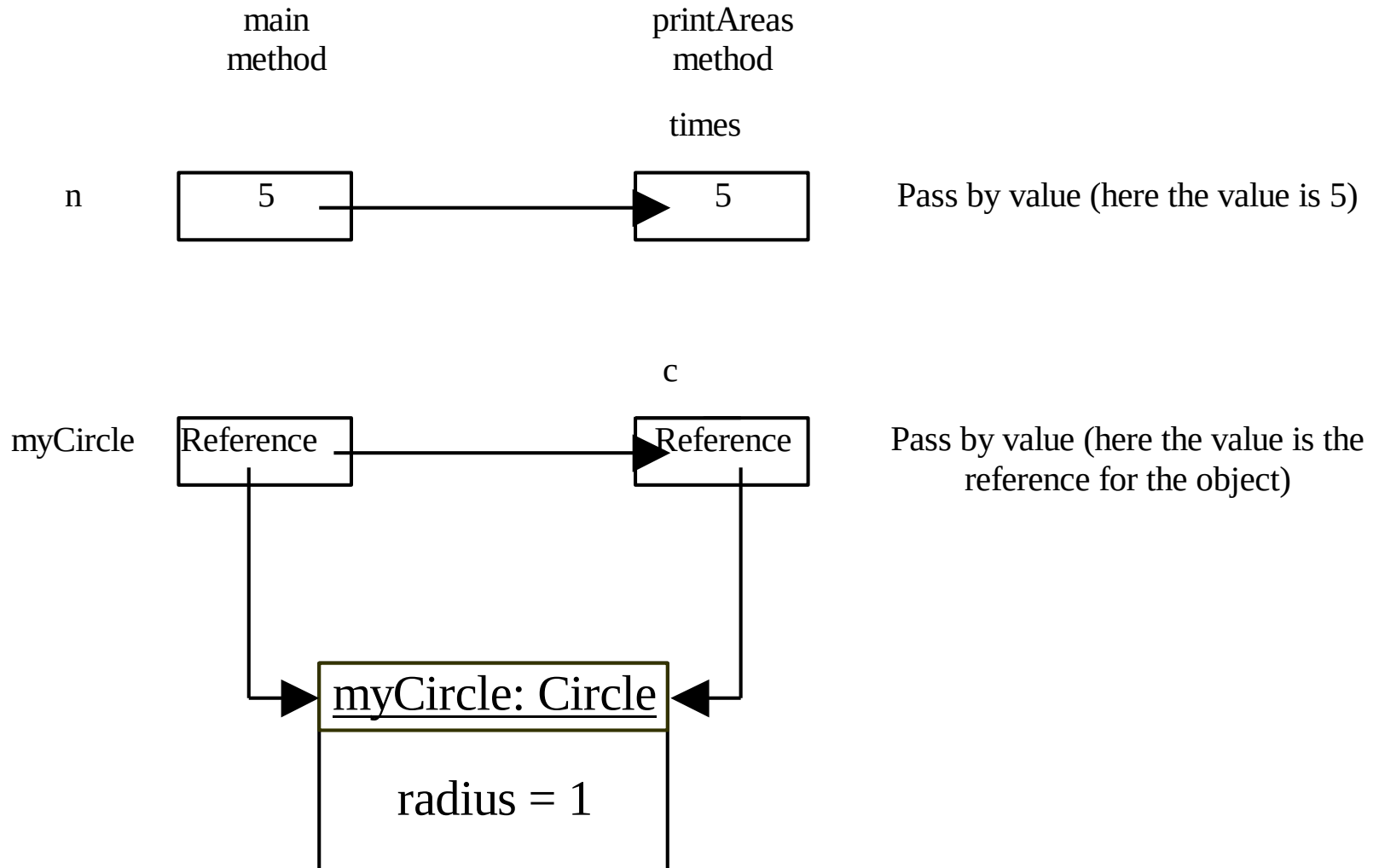
```
public class TestPassObject {  
    /** Main method */  
    public static void main(String[] args) {  
        // Create a Circle object with radius 1  
        Circle3 myCircle = new Circle3(1);  
  
        // Print areas for radius 1, 2, 3, 4, and 5.  
        int n = 5;  
        printAreas(myCircle, n);  
  
        // See myCircle.radius and times  
        System.out.println("\n" + "Radius is " + myCircle.getRadius());  
        System.out.println("n is " + n);  
    }  
  
    /** Print a table of areas for radius */  
    public static void printAreas(Circle3 c, int times) {  
        System.out.println("Radius \t\tArea");  
        while (times >= 1) {  
            System.out.println(c.getRadius() + "\t\t" + c.getArea());  
            c.setRadius(c.getRadius() + 1);  
            times--;  
        }  
    }  
}
```

Passing Objects to Methods

```
▶ public class Circle3 {  
    /** The radius of the circle */  
    private double radius = 1;  
  
    /** The number of the objects created */  
    private static int numberOfObjects = 0;  
  
    /** Construct a circle with radius 1 */  
    public Circle3() {  
        numberOfObjects++;  
    }  
  
    /** Construct a circle with a specified radius */  
    public Circle3(double newRadius) {  
        radius = newRadius;  
        numberOfObjects++;  
    }  
  
    /** Return radius */  
    public double getRadius() {  
        return radius;  
    }  
  
    /** Set a new radius */  
    public void setRadius(double newRadius) {  
        radius = (newRadius >= 0) ? newRadius : 0;  
    }  
  
    /** Return numberOfObjects */  
    public static int getNumberOfObjects() {  
        return numberOfObjects;  
    }  
  
    /** Return the area of this circle */  
    public double getArea() {  
        return radius * radius * Math.PI;  
    }  
}
```



Passing Objects to Methods



ตัวแปรของวัตถุและเมธอดของวัตถุ (Instance Variables and Methods)

- ▶ ตัวแปรของวัตถุ(Instance variables)
 - ▶ จะเป็นตัวแปรที่อยู่ภายในวัตถุ
 - ▶ ส่วนเมธอดของวัตถุจะถูกเรียกโดยวัตถุของคลาส
 - ▶ โดยทั้งตัวแปรวัตถุ และเมธอดของวัตถุจะต้องสร้างวัตถุขึ้นมา ก่อนจึงจะสามารถใช้ได้



ตัวแปรของคลาส ค่าคงที่ และเมธอดของคลาส(Class Variables, Constants, and Methods)

- ▶ Class variables จะถูกใช้งานได้โดย instance ของคลาสทุกๆ instance เป็นตัวแปรที่ไม่จำเป็นต้องสร้างอินสแตนซ์ของวัตถุขึ้นมาก่อน
- ▶ Class methods ไม่ถูกผูกกับวัตถุใด ๆ
- ▶ Class constants เป็น final variables ที่จะแชร์โดย instance ทุก ๆ instances ของคลาส
- ▶ เราสามารถกำหนด class variables, constants และ methods ได้โดยใช้คำ static



Class Variables, Constants, and Methods.

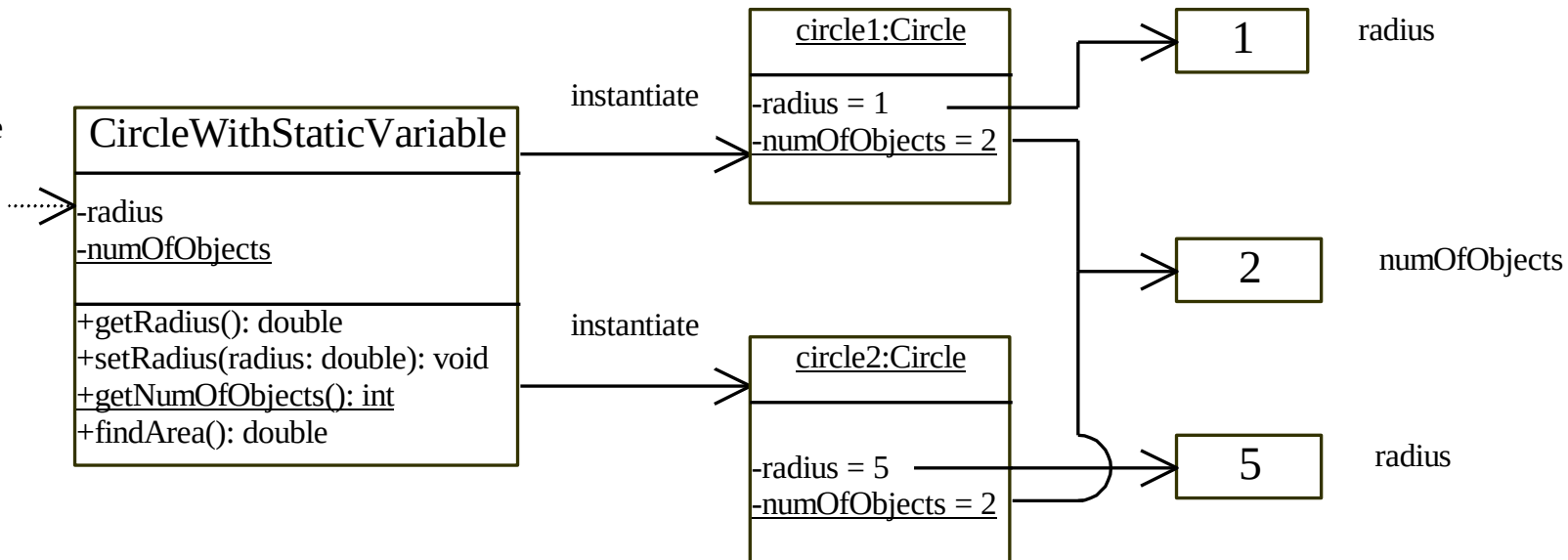
UML Notation:

+: public variables or methods

-: private variables or methods

underline: static variables or methods

radius is an instance variable, and numObjects is a class variable



Example

Using Instance and Class Variables and Method

```
▶ public class TestCircleWithStaticMembers {  
    /** Main method */  
    public static void main(String[] args) {  
        System.out.println("Before creating objects");  
        System.out.println("The number of Circle objects is " +  
            CircleWithStaticMembers.numberOfObjects);  
  
        // Create c1  
        CircleWithStaticMembers c1 = new CircleWithStaticMembers();  
  
        // Display c1 BEFORE c2 is created  
        System.out.println("\nAfter creating c1");  
        System.out.println("c1: radius (" + c1.radius +  
            ") and number of Circle objects (" +  
            c1.numberOfObjects + ")");  
  
        // Create c2  
        CircleWithStaticMembers c2 = new CircleWithStaticMembers(5);  
  
        // Modify c1  
        c1.radius = 9;  
  
        // Display c1 and c2 AFTER c2 was created  
        System.out.println("\nAfter creating c2 and modifying c1");  
        System.out.println("c1: radius (" + c1.radius +  
            ") and number of Circle objects (" +  
            c1.numberOfObjects + ")");  
        System.out.println("c2: radius (" + c2.radius +  
            ") and number of Circle objects (" +  
            c2.numberOfObjects + ")");  
    }  
}
```



Example

Using Instance and Class Variables and Method

```
public class CircleWithStaticMembers {  
    /** The radius of the circle */  
    double radius;  
  
    /** The number of the objects created */  
    static int numberOfObjects = 0;  
  
    /** Construct a circle with radius 1 */  
    CircleWithStaticMembers() {  
        radius = 1.0;  
        numberOfObjects++;  
    }  
  
    /** Construct a circle with a specified radius */  
    CircleWithStaticMembers(double newRadius) {  
        radius = newRadius;  
        numberOfObjects++;  
    }  
  
    /** Return numberOfObjects */  
    static int getNumberOfObjects() {  
        return numberOfObjects;  
    }  
  
    /** Return the area of this circle */  
    double getArea() {  
        return radius * radius * Math.PI;  
    }  
}
```



ขอบเขตของตัวแปร

- ▶ ขอบเขตของ Instance variable และ class variable จะสามารถใช้ได้ทั้งคลาส และสามารถที่จะประกาศที่ตำแหน่งใด ๆ ก็ได้ภายในคลาส
- ▶ ขอบเขตของตัวแปรที่เป็นแบบท้องถิ่น Local variable จะเริ่มจาก ณ ตำแหน่งที่ประกาศและต่อเนื่องไปจนกระทั่งสิ้นสุดบล็อกที่เก็บตัวแปรนั้น โดยตัวแปรแบบท้องถิ่นจะต้องประกาศก่อนถึงจะใช้งานได้
- ▶ การใช้คีย์เวิร์ด **this** ใช้สำหรับอ้างถึงวัตถุปัจจุบัน นอกจากนี้ยังสามารถใช้เรียก constructors ของวัตถุได้



การเขียนคำสั่งเพื่อรับค่าในรูปแบบ dos mode

► การรับค่าโดยใช้ เมธอดในคลาส **Scanner**

```
import java.util.Scanner;

public class ComputeLoan {
    public static void main(String[] args)
    {
        // Create a Scanner
        Scanner input = new
        Scanner(System.in);

        // Enter yearly interest rate
        System.out.print("Enter yearly
        interest rate, for example 8.25: ");
        double annualInterestRate =
        input.nextDouble();

        // Obtain monthly interest rate
        double monthlyInterestRate =
        annualInterestRate / 1200;

        // Enter number of years
        System.out.print(
        "Enter number of years as an
        integer, for example 5: ");
```

```
        int numberOfYears = input.nextInt();

        // Enter loan amount
        System.out.print("Enter loan
        amount, for example 120000.95: ");
        double loanAmount =
        input.nextDouble();

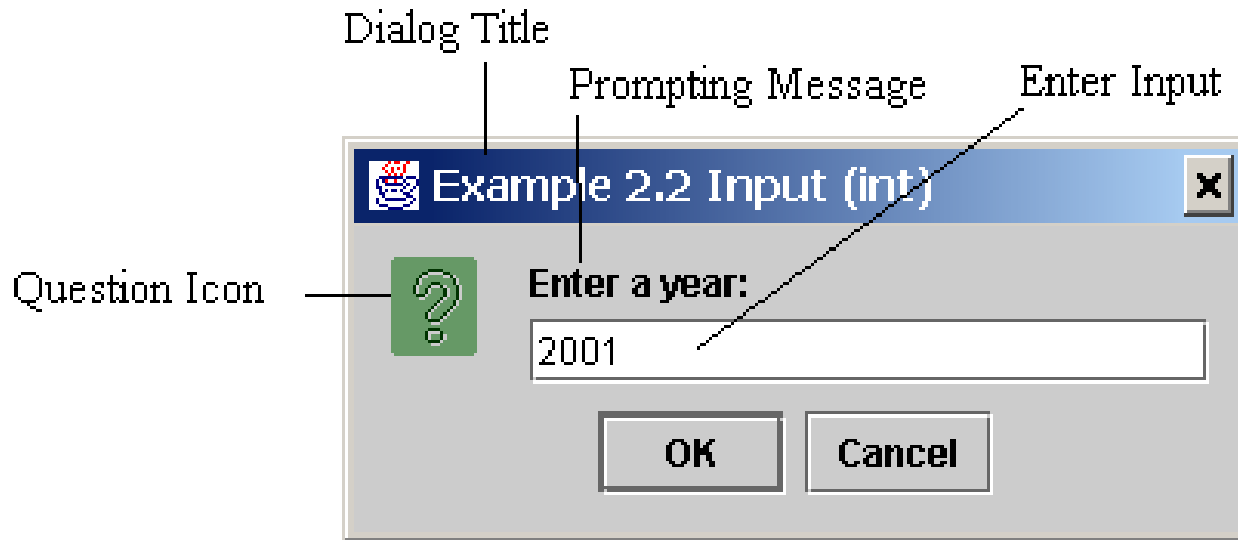
        // Calculate payment
        double monthlyPayment = loanAmount
        * monthlyInterestRate / (1
        - 1 / Math.pow(1 +
        monthlyInterestRate, numberOfYears *
        12));
        double totalPayment =
        monthlyPayment * numberOfYears * 12;

        // Format to keep two digits after
        the decimal point
        monthlyPayment =
        (int)(monthlyPayment * 100) / 100.0;
        totalPayment = (int)(totalPayment
        * 100) / 100.0;

        // Display results
        System.out.println("The monthly
        payment is " + monthlyPayment);
        System.out.println("The total
        payment is " + totalPayment);
    }
}
```

Getting Input from Input Dialog Boxes

```
String string = JOptionPane.showInputDialog(  
    null, "Prompt Message", "Dialog Title",  
    JOptionPane.QUESTION_MESSAGE);
```



Java API and Core Java classes

<code>java.lang</code>	Contains core Java classes, such as numeric classes, strings, and objects. This package is implicitly imported to every Java program.
<code>java.awt</code>	Contains classes for graphics.
<code>java.applet</code>	Contains classes for supporting applets.
<code>java.io</code>	Contains classes for input and output streams and files.
<code>java.util</code>	Contains many utilities, such as date.
<code>java.net</code>	Contains classes for supporting network communications.

<http://docs.oracle.com/javase/7/docs/api/index.html>

<http://docs.oracle.com/javase/tutorial/index.html>